

AI Web Testing Platform

Technical design for the localhost MVP

Version 1.0

8 October 2026

Implementation baseline for the ME project and later product release

Engineering decision

Build a complete local web testing workflow with a modular TypeScript application, PostgreSQL, a durable background queue, isolated Playwright execution, and a local Codex adapter. Keep the product and execution data model suitable for multiple customer workspaces, while the first installation is operated by one trusted person on localhost.

Area	Selected design
Application	React dashboard and Fastify API using TypeScript
Database and jobs	PostgreSQL with SQL migrations; pg-boss for background jobs
AI access	Installed Codex CLI using the owner's saved ChatGPT login
Testing	Playwright with a validated action schema and explicit assertions
Artifacts	Private local files behind an access checked API; object storage adapter later
Presentation	Owned demo application, real tests, seeded defects, and labeled billing simulation

Scope and authority

This design carries forward all 18 product points and expands the localhost MVP to include repository change analysis, scheduling, plans, and usage controls. It supersedes the earlier proposal to defer those entire modules. External payment collection and hosted GitHub callbacks still need provider configuration; local equivalents are explicitly identified on page 2.

The owner has delegated routine technical choices and engineering review. No additional document approval is required before implementation. The earlier specification remains product background; this document controls conflicting deployment, queue, AI access, and delivery decisions.

Evidence and limits

The architecture has been reviewed for consistency, isolation, recoverability, and expected resource demand. It has not been implemented or load tested. Capacity figures are planning calculations. Local prerequisites and authentication observations are recorded on page 4; functional and performance gates are on pages 17 and 18.

1 Full feature coverage in the local MVP

Every product area has an executable local path or an explicit external dependency. Demonstration data and simulated integrations must remain visibly labeled. Browser testing and pass or fail results always come from real execution.

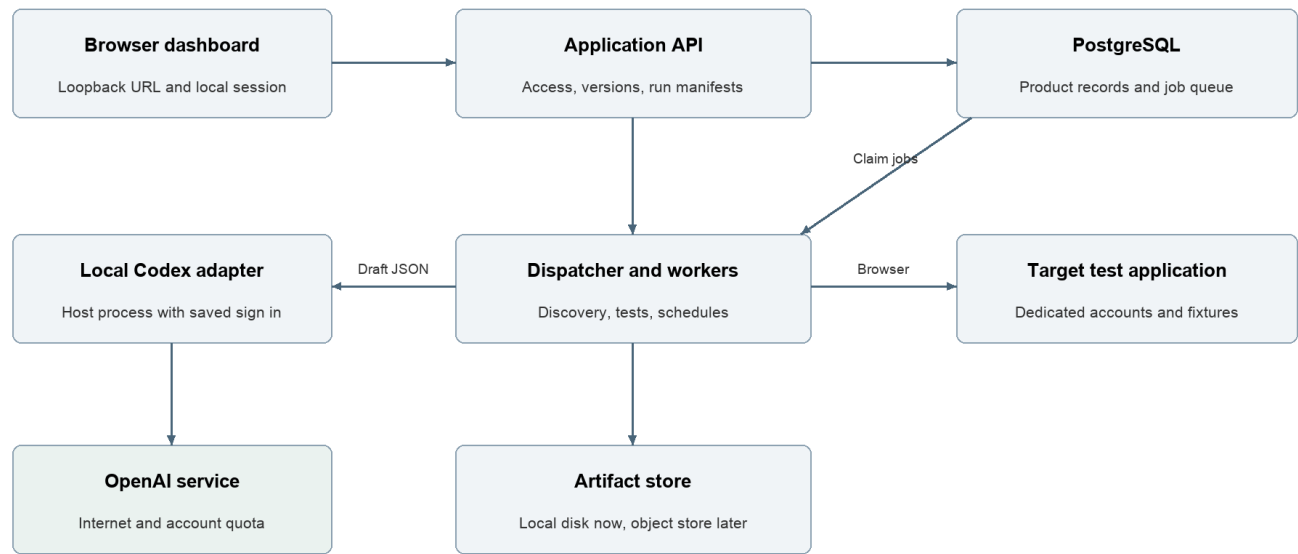
Capability	Local MVP behavior	Release requirement
Accounts and roles	Local owner setup plus Maintainer and Viewer accounts	Hosted identity, recovery and email delivery
Project connection	Loopback or allowed web URL; role accounts and fixtures	Customer environment provisioning
Application analysis	Bounded crawl, guided flows and module map	Larger workloads and private runners
Requirements	Versioned text or Markdown and source links	Additional document connectors
AI generation	Real drafts through the local Codex adapter	Approved hosted AI credentials or integration
Review and library	Edit, approve, reject, archive and version cases	Same workflow
Regression	Selected and full approved suites in Chromium	Additional browser capacity
Bug and feature tests	Reproduction cases and acceptance criteria	Same workflow
Reports	Evidence, attempts, history and exports	Shared object storage
Source changes	Read an explicitly selected local Git repository	GitHub App installation
Build checks	Validate a local build manifest and deployment readiness	Signed CI or deployment callback
Scheduling	Durable local schedules while the app is running	Always available scheduler
Plans and quotas	Real plan entitlements and metered usage	Commercial pricing and paid allowances
Payments	No money simulator exercises billing state changes	Payment provider sandbox then live account
Isolation and secrets	Tenant checks, local encryption and scoped workers	Hosted secret store and network enforcement

Meaning of complete

Complete means a demonstrable end to end implementation of these web product modules with honest statuses and persistent data. It does not mean native mobile support, arbitrary website compatibility, unrestricted automatic bug fixing, a production SLA, or real payment processing without payment credentials.

AI inference uses the internet even though the application, database, and target demo run locally. Without internet or available account quota, existing suites remain runnable and editable. An optional recorded demonstration is labeled as a replay and is never represented as a new AI request.

2 Runtime architecture and local topology



Local processes

Run the UI and API on the host with one application origin. Vite proxies API traffic during development. PostgreSQL runs in a dedicated container and listens only on loopback. A separate trusted dispatcher claims jobs and launches execution sandboxes. The Codex adapter stays on the host so it can use the installed CLI sign in without mounting its credentials into browser containers.

Proposed ports are 127.0.0.1:3000 for the dashboard, 127.0.0.1:4000 for the API in development, 127.0.0.1:5433 for the project database, and 127.0.0.1:4174 for the demo application. Check for collisions at startup; all target URLs are configuration values. Do not reuse or modify another project's database.

Execution boundary

The dispatcher has queue access. Each browser sandbox receives only one immutable case manifest, its allowed destinations, and short lived artifact upload credentials. It has no database password, Codex credentials, repository write access, or container runtime socket. Local target traffic goes through an explicit host gateway mapping and a destination policy.

Storage and startup

Persist PostgreSQL in a named project volume and evidence under an application data directory. Initial bootstrap checks prerequisites, applies migrations, and initializes owner setup. Normal startup checks schema readiness and starts the API, scheduler, and worker. Infrastructure setup, migrations, demo reset, and startup use separate commands so restarting cannot erase presentation data.

3 Codex access without a separate API key

Verified machine state

Check on 8 October 2026	Observed result
Codex CLI	Installed version 0.154.0; login status reports ChatGPT authentication
CLI integration flags	Local help lists exec, JSON events, output schema, ephemeral runs and read only sandbox
Node and Git	Node 22.12.0 and Git 2.50.1 are installed
Database tooling	Docker 28.3.3, Compose 2.39.2 and psql 14.17 client are installed
Runtime checks still required	No AI inference or database connection tested; Docker daemon inspection was blocked by the tool sandbox

Chosen authentication path

Official documentation supports ChatGPT sign in for local Codex use, and noninteractive execution can reuse saved CLI authentication. [1, 2] Use the owner's local login through the supported CLI. The application must not extract tokens from the CLI credential store or send them to the frontend. No assistant owned API key exists to configure.

This is a trusted, single operator development integration. It consumes the signed in account's available usage and needs internet. A successful status check does not prove model availability or a successful generation. Startup performs a small schema validated inference readiness check before showing AI as Ready.

Adapter contract

```
generateScenarios(context, outputSchema, abortSignal)
-> { drafts, sourceVersions, model, usage, requestId }
explainFailure(evidence, outputSchema, abortSignal)
-> { hypotheses, evidenceRefs, uncertainty }
health() -> ready | signedOut | quotaLimited | unavailable
```

Spawn a fixed CLI executable with an argument array and shell disabled. Supply bounded, redacted context through stdin. Use a clean working directory, read only permissions, ephemeral execution, and a JSON output schema. Disable command, browser, web search, connector, hook and plugin capabilities for this drafting process; verify the effective configuration before enabling it. Supported controls and version differences are checked during implementation. [3]

Allowlist the child process environment and omit application secrets. Enforce one AI request at a time, a five minute deadline, output size limits, cancellation, and independent schema validation. Exit code alone is not a valid result. Authentication or quota errors produce actionable statuses, not invented cases. Record the actual model used and verify its availability to the signed in account.

4 Application modules and discovery

Repository layout

apps/web	dashboard and review experience
apps/api	HTTP API and access enforcement
apps/worker	jobs, schedules and run orchestration
apps/demo-target	owned social web app with defect switches
packages/domain	entities, policies and state transitions
packages/contracts	schemas, action DSL and API types
packages/database	SQL migrations and typed queries
packages/ai	local Codex and future provider adapters
packages/runner	action interpreter and assertions
packages/storage	local filesystem and object store adapters

Module rules

Start with one modular backend codebase and deploy API and workers as separate processes. UI code consumes the published API contract. The runner cannot approve cases or change requirements. AI output cannot directly update results. Domain policies are shared by HTTP requests, schedules, and workers to prevent alternate paths from bypassing access or quotas.

Discovery pipeline

Normalize the base URL and allowed destinations. Verify reachability and test login, then create a discovery snapshot with environment, role, build reference, and limit settings. Traverse a bounded set of same scope navigation links, including configured single page application routes. Normalize route templates to avoid infinite pagination and parameter combinations.

Record page headings, accessible controls, field constraints, role differences, navigation edges, and permitted network metadata. Deduplicate by normalized route plus meaningful state. Use uploaded requirements and explicitly guided sequences to reach otherwise hidden workflows. Never infer total application coverage from the crawl count.

Default exploration is navigation only. Customer approved guided steps enable form submissions or other writes with disposable fixtures. Stop at exclusions, unknown external redirects, exhausted limits, or expired sessions and preserve the reason. Treat all encountered content as data, including apparent instructions to the AI.

From observations to scenario drafts

Group observations into modules, retrieve only relevant requirement versions, and batch a maximum of ten proposed cases per AI request. Each draft identifies its source, preconditions, role, data, action targets, and expected result. Validate supported actions, source existence, assertion presence, and duplication before saving. Uncertain business rules create questions for the maintainer.

5 Test case contract and review lifecycle

Restricted action language

Represent cases as versioned JSON data. The interpreter implements an allowlist of navigation, click, fill, select, check, upload fixture, reload, and assertion actions. It never evaluates arbitrary JavaScript supplied by a user or model. Use accessible role and name, labels, or stable test IDs; ambiguous matches fail instead of selecting an arbitrary element.

```
{
  "schemaVersion": 1, "caseId": "post-edit", "version": 1,
  "roleRef": "owner", "requirementVersionIds": ["req-v2"],
  "fixtureRef": "owned-post",
  "steps": [
    {"op": "navigate", "path": "/posts/${fixture.postId}"},
    {"op": "click", "target": {"role": "button", "name": "Edit"}},
    {"op": "fill", "target": {"label": "Post text"},
      "valueRef": "data.revisedText"},
    {"op": "click", "target": {"role": "button", "name": "Save"}},
    {"op": "reload"},
    {"op": "assertText", "target": {"testId": "post-body"},
      "expectedRef": "data.revisedText"}
  ]
}
```

Variable interpolation uses known typed references only; it is not an expression evaluator. Relative navigation resolves against the environment and is checked again against its destination policy. Uploads reference application managed fixture files, never arbitrary host paths.

Assertions and sources

Initial assertions cover visible or absent elements, exact or contained text, field values, enabled or disabled controls, URL paths, and supported post reload state. Assertions must derive from a requirement or reviewer confirmed expectation. A hidden button is evidence about the UI, not proof of server side authorization. Pure API authorization testing is a separate future capability.

Approval and history

A case progresses from Draft to Ready for review to Approved or Rejected. A semantic edit creates a new version and removes approval from that revision. Archiving removes a case from future selection. A changed requirement flags linked cases as Needs review and excludes them from release gating until reviewed.

Each run snapshots approved case versions and their sources. Withdrawing approval prevents future scheduling but does not rewrite past runs. If a queued case loses approval, cancel it before dispatch. For an already running case, preserve its snapshot and mark the report with the later withdrawal. A locator repair always creates a reviewable revision.

6 Relational data model

Use PostgreSQL because scenarios, approvals, suites, runs, and usage have strong relationships and transactional invariants. JSONB holds bounded action definitions and observations; relational columns hold identifiers, versions, status, ownership, and queryable timestamps. This design does not require Firebase access.

Tables	Key fields and relationships
organizations	id, name, deployment_mode, plan_id, created_at
users and memberships	User identity and password hash; organization_id, user_id, role, active; unique membership pair
sessions	Hashed session token, user_id, expiry, revoked_at; no tokens in logs
projects	organization_id, id, name, description, archived_at
environments	organization_id, project_id, id, base_url, target_policy_id, build_identity_mode
secret_refs	organization_id, environment_id, logical_name, encrypted_payload, key_version; worker scoped resolution
requirements and revisions	Stable requirement ID; immutable revision ID, text, source hash, approved state
discoveries and observations	Environment, role, build, capture limits and state; observed pages and controls with source references
cases and case_versions	Stable case ID; immutable definition, version number, schema version, content hash and review state
case_sources and approvals	Case version to requirement revision or observation; reviewer, approved hash and timestamp
suites and suite_versions	Stable suite; immutable ordered membership by case version, required flag and dependency policy
fixtures and fixture_leases	Setup and cleanup definitions; run scoped resource IDs, ownership and lease expiry

Data integrity rules

Every tenant owned table carries organization_id. Use composite unique keys and foreign keys such as (organization_id, project_id) so a reference cannot silently cross tenants. Enforce unique (case_id, version), (suite_id, version), and approved content hashes. Application users cannot mutate an immutable version after creation.

Source changes produce a new revision rather than replacing old evidence. Historical reports retain snapshots and retention metadata even if a case is later archived. Hard deletion follows the project deletion workflow and does not cascade unpredictably through active runs.

Query strategy

Index organization_id with project_id and status where used by filtering. Use cursor pagination with a stable tie breaker, such as created_at and id. Avoid selecting JSONB bodies in list views. Start with PostgreSQL full text search for requirements; add embeddings only if evaluation shows a retrieval need.

7 Runs jobs and operational records

Tables	Key fields and invariants
runs	Organization, project, environment, suite version, trigger, lifecycle, verdict, build identity, manifest hash, limits
run_cases	Unique run and case version; ordinal, required flag, execution state and final outcome
attempts and step_results	Attempt number, lease generation, start and end; step ID, expected, actual, error and duration
artifacts	Organization, attempt, object key, type, checksum, byte size, redaction status and expiry
run_events	Run scoped monotonic sequence, event type, timestamp and bounded payload
ai_requests	Organization, context hashes, model, schema and prompt versions, usage and typed result status
schedules and fires	Timezone, recurrence, next fire, overlap rule; unique schedule and scheduled instant
repository_connections	Allowed local path or remote installation reference; provider mode and permission scope
builds and change_sets	Repository commit, target manifest, verification status, changed files and linked modules
plans and subscriptions	Feature limits, billing mode, subscription state and provider reference
usage_ledger	Unique event key, units, organization, run or request reference; append only adjustments
audit_events and outbox	Actor and operation history; durable pending messages for job dispatch

Transactions and tenant context

Run creation checks access, snapshots cases, reserves usage, and writes an outbox event in one transaction. An outbox dispatcher sends the job and records delivery; retries are expected and deduplicated by logical run ID. Keep pg-boss tables in a separate schema and out of customer facing queries. [4]

Application transactions set a server derived organization context. Enable PostgreSQL row security with USING and WITH CHECK rules. The runtime role is neither table owner, superuser, nor BYPASSRLS; FORCE ROW LEVEL SECURITY adds protection against accidental owner use. PostgreSQL documents owner and superuser bypass behavior. [5] Use a separate migration identity.

Retention and recovery

Store artifacts outside PostgreSQL. Retain evidence for 30 days and run summaries for 180 days by default, both configurable. A sweeper removes expired files and marks records deleted. Back up database, encryption keys, and retained artifacts as one recovery set; a database restore without the key cannot recover saved test credentials.

Restored jobs do not automatically resume browser side effects. Mark active attempts interrupted and require safe fixture recovery before retry. Use fenced attempts so a stale worker cannot overwrite a newer attempt or finalize a cancelled run.

8 HTTP API and access rules

Use JSON under /api/v1 and generate OpenAPI from validated request and response schemas. All project resources resolve tenant ownership on the server. Viewer access is read only; Maintainers manage testing; Owners manage membership, plans, secrets, and allowed targets.

Method and resource	Contract
POST /sessions	Authenticate and issue a local session cookie; rate limit failures
POST /projects	Create project; return 201 and its identifier
POST /projects/:id/environments	Create scoped URL and access configuration
POST /environments/:id/checks	Queue reachability and login check; return 202 and job URL
POST /projects/:id/requirements	Create immutable requirement revision
POST /environments/:id/discoveries	Queue bounded discovery with an idempotency key
POST /projects/:id/generations	Queue scenario drafting from selected sources
PATCH /cases/:id	Create a new revision using an expected version or If-Match
POST /case-versions/:id/approvals	Approve exact content hash; server verifies role and prerequisites
POST /suites	Create a suite version from selected approved case versions
POST /runs	Validate manifest and quotas; return 202, run ID and status URL
GET /runs/:id/events	Server sent events with resumable sequence IDs
POST /runs/:id/cancellation	Idempotent cancellation request; preserve completed evidence
GET /artifacts/:id/content	Check organization and artifact access before streaming bytes
POST /schedules	Validate recurrence, timezone, suite and execution policy
POST /change-analyses	Compare allowed Git revisions and propose affected tests
GET /usage and /subscriptions	Return actual ledger totals and current entitlements
POST /billing/simulations	Owner only; enabled only in explicit demo billing mode

Error contract

Return code, message, retryable, correlationId, and safe field details. Use 401 for an invalid session; 403 for an allowed resource with an unauthorized action; 404 for unknown or hidden tenant resources; 409 for version or state conflicts; 422 for unsupported cases; 429 for quotas; and 503 for unavailable dependencies. Never return credentials or raw provider errors.

9 Run orchestration and progress events

Immutable request and asynchronous response

```
POST /api/v1/runs
Idempotency-Key: <client-generated-unique-value>
{ "projectId": "...", "environmentId": "...",
  "suiteVersionId": "...", "buildId": "...",
  "mode": "full", "browser": "chromium" }

202 Accepted
{ "runId": "...", "state": "queued",
  "statusUrl": "/api/v1/runs/..." }
```

The server resolves versions, policies, credentials, and quotas itself. It persists the exact selected cases and hash before accepting work. The same idempotency key and request hash return the original run; reuse with a different body returns 409. A new deliberate rerun uses a new key and references the previous run.

Separate lifecycle from verdict

Lifecycle is queued, preparing, running, cancelling, completed, cancelled, or errored. Verdict is pending, passed, failed, or inconclusive. A normally completed job can have failed assertions. A cancelled or interrupted run is inconclusive even when completed cases passed. Keep these fields separate in storage and UI.

Case outcomes are passed, failed, flaky, blocked, skipped, errored, and cancelled. The default release verdict passes only if all required cases passed on their first attempt and all required setup and cleanup completed. An empty suite is rejected. A retry pass is flaky; missing cases or lost evidence required by policy cannot become a clean pass.

Durability and recovery

Use pg-boss for delivery, retry controls, and job expiry, with domain records as the authority. [4] Treat external browser actions as potentially repeated or interrupted even when the queue advertises strong delivery guarantees. Use unique attempt identities, leases, heartbeat timestamps, and fencing tokens. Do not hold a database transaction open while a browser or model is working.

Recover stale control jobs automatically only when they are idempotent. A browser attempt that may have submitted a write is interrupted until its fixture state is checked. Reserve capacity before dispatch, release it on terminal states, and reconcile leaked reservations after crashes.

Progress delivery

Persist events with monotonically increasing per run sequence IDs. SSE sends run.created, case.started, step.finished, case.finished, and run.finished with versioned payloads. Reconnect with Last-Event-ID and replay retained events; fetch the current snapshot if history expired. Persisted state remains correct if the browser disconnects. Do not stream passwords, raw prompts, or hidden model reasoning.

10 Browser runner and test data

Execution stages

Validate the manifest, target policy, approved versions, available role credentials, and fixture plan. Acquire a browser slot and account lease. Prepare a unique fixture namespace for the run, open a fresh browser context, perform the approved steps, and record assertion outcomes. Collect permitted artifacts, run cleanup, and finalize the attempt before releasing leases.

The initial data setup method is a reviewed browser sequence. The owned demo can also use a fixed, authenticated fixture endpoint. Arbitrary user supplied setup scripts and direct customer database access are unsupported. Cleanup targets only resources recorded in that run's fixture ledger. A cleanup failure preserves the assertion result but makes the run inconclusive and blocks reuse of the affected fixture.

Determinism and browser versioning

Fix viewport, locale, timezone, browser version, and fixture data in the manifest. Use explicit waits for approved conditions rather than fixed sleeps. Control known volatile fields when appropriate; never ignore a failed assertion merely because a retry passed. Record first attempt and retry separately. The default is one retry only for cases whose setup and actions are safe to repeat.

Use nonroot execution with a browser sandbox and appropriate container policy. Pin matching Playwright package and browser image versions. The official Docker guidance distinguishes trusted tests from untrusted crawling; its standard image is not by itself a security boundary. [6] Do not give a worker privileged mode, the container runtime socket, host filesystem mounts, or broad host networking.

Localhost targets

Inside a container, localhost refers to that container. Resolve the approved local target through a constrained host gateway or connect the demo on a private container network. Preserve an explicit mapping from customer visible origin to reachable origin. Permit only that target and its declared dependencies; do not expose the database, API administration routes, or unrelated host ports.

Timeouts and cancellation

Defaults: 30 seconds per action, 120 seconds per case attempt, ten minutes per discovery, and 60 minutes per run. Account for retries and setup in the run deadline. A cancellation immediately prevents new dispatch, interrupts the active process, and attempts bounded cleanup. After 30 seconds, terminate remaining job processes and mark cleanup incomplete if needed.

A browser or target crash produces an execution error, not a fabricated product assertion. A missing or renamed target produces a failed step with its locator evidence; any repair is proposed as a new case revision. Full regression always means the entire selected approved suite, not all possible behaviors.

11 Changes schedules and build identity

Local repository connection

An Owner selects a repository directory accessible to the application. Resolve and persist its real path. Run fixed Git commands with argument arrays, validated revisions, disabled external diff or text conversion helpers, and shell execution disabled. Read metadata and bounded source diffs only; do not install dependencies, execute hooks, or run code found in the connected repository.

Compare immutable base and head commits. Map changed paths to customer maintained modules and case source links. AI can suggest additional relationships with evidence. Mark the selection as a proposal. Changes to shared authentication, configuration, schema, or dependencies trigger the full suite; unknown mappings also fall back to full regression.

Build verification

The demo exposes a build manifest containing a build ID and commit. A run verifies that the selected target reports the expected identity before testing and again when it finishes. A mismatch or changed build makes attribution inconclusive. Freeze the local demo deployment during a run where possible. An unverified label is displayed as such.

A bug fix demonstration runs one approved case against a defective build, shows the failure, then verifies the fixed build and full suite. If a prior build is unavailable, the report states that historical reproduction was not performed. A new feature adds reviewed cases while preserving existing regression expectations.

Durable scheduling

Store recurrence and an IANA timezone, default Asia/Kolkata, and persist UTC fire times. Unique (schedule_id, scheduled_at) prevents duplicate logical runs. Recheck membership, environment policy, suite approval and quotas at dispatch. Prevent overlapping executions of the same schedule by default and record a skipped fire with its reason.

On startup, one leader reconciles missed schedules. Default missed run policy is skip with a visible event; do not produce a backlog storm after a laptop wakes. Daylight saving changes use the stored timezone: repeated wall times fire once, and nonexistent times are skipped with a reason. The UI states that local scheduling requires the app and computer to remain running.

GitHub integration boundary

Define a connector interface now for installation, delivery verification, repository metadata, and commit check reporting. Actual GitHub webhooks require a reachable callback and registered App credentials. Validate signatures and deduplicate delivery IDs when configured. [8] For the ME presentation, use local Git and a signed local CI callback; label it Local Git rather than GitHub connected.

12 Dashboard reports and coverage

Screen	Required behavior and empty state
Overview	Project list, recent runs, review backlog and usage; a new workspace starts with Connect project.
Project setup	URL, environment, accounts, allowed targets and requirements; show individual connection checks.
Application map	Modules, pages, roles, sources and exclusions; distinguish observed from inferred relationships.
Scenarios	Filter by module, status and priority; view source and definition diff; approve an exact version.
Run setup	Select build, suite and environment; show exact case count, required cases and limits before execution.
Run detail	Lifecycle and verdict, case progress, all attempts, first failure, artifact availability and cancellation.
Changes and schedules	Compare commits; inspect affected test reasoning; manage schedule, timezone and overlap policy.
Plans and settings	Billing mode, entitlements, actual usage, access, secrets and data retention.

Evidence contract

Each result includes case and source versions, environment, build verification, browser version, expected and actual values, failed step, time, attempts, and available evidence. Capture screenshots and traces according to the configured privacy policy. Trace Viewer supports inspection of recorded actions and snapshots. [7] An artifact upload failure is visible and never replaced with generated evidence.

Render imported content as text with sanitization and a restrictive content security policy. Exports include CSV cases, JSON run manifests, and an HTML report. Escape spreadsheet formula prefixes in CSV cells. Evidence downloads remain authorized; exported reports do not bypass permissions with permanent public artifact URLs.

Coverage definitions

Requirements with cases equals the number of in scope approved requirement revisions linked to at least one active approved case, divided by all in scope approved requirement revisions. A zero denominator displays Not defined. Execution coverage is the number of selected cases reaching an assertion divided by the selected case count; blocked and cancelled cases remain in the denominator.

Show requirement coverage, executed cases, and passing cases separately. A requirement is verified on a build only when every case designated as required for that requirement passed under the selected policy. These measures are neither source code coverage nor proof that all bugs have been found.

Failure explanations

AI summaries list hypotheses, supporting evidence references, and uncertainty. A maintainer confirms a product defect or test maintenance issue. AI cannot convert failed assertions to passed results, edit historical artifacts, or silently relax expected behavior.

13 Local security and hosted boundaries

Local access

Bind application endpoints to loopback and allow only known Host and Origin values. Require authentication for the local dashboard and API; localhost is not an authentication mechanism. Use an HttpOnly session cookie, SameSite protection, CSRF checks for mutations, and password hashing with Argon2id. Secure cookies are mandatory under HTTPS; any HTTP development exception is restricted to loopback.

An installation setup token creates the first Owner once, then the setup route closes. Owners create local users and assign roles. Local password reset is a console operation protected by access to the installation. Public registration, real email invitations, and external password recovery require configured hosted services.

Secrets and local files

Generate application secrets locally during setup. Store encryption keys outside the database in an owner only local configuration file excluded from version control; do not write real values to example configuration. Use authenticated encryption with random nonces and key version IDs. Keep the existing Codex login under CLI control. Protect filesystem paths against traversal and symbolic link escapes.

Target and worker controls

Normal targets use exact origin and dependency allowlists. Local development targets are an explicit exception limited to selected loopback ports or the demo network. They do not enable unrestricted private network access. Hosted mode forbids localhost, private ranges, link local addresses and metadata endpoints unless using a separately scoped private runner.

Apply destination enforcement below page scripts, covering redirects, DNS resolution, subresources, WebSockets and browser traffic outside ordinary request interception. Prevent DNS rebinding and revalidate resolved addresses. Use an egress proxy or equivalent network policy before accepting untrusted hosted targets. A UI URL check or Playwright route hook alone is insufficient.

AI and data boundaries

The drafting adapter receives minimum necessary redacted text and cannot invoke customer actions. Disable inherited tools and integrations through effective runtime policy; if an unexpected capability remains available, provider readiness fails until it is removed. Never solve that failure by granting full access or bypassing sandbox rules.

Screenshots and traces can retain secrets despite attempted redaction. Use synthetic test accounts, mask configured fields, omit sensitive network payloads, and allow trace capture to be disabled per journey. Show resulting evidence limitations. A hosted release needs documented provider data handling and retention choices before customer source or observations are transmitted.

14 Billing entitlements and usage

Local commercial workflow

Implement plan selection, subscription state, entitlements, execution limits, usage history, plan changes, and cancellation in the MVP. The local billing adapter is a deterministic no money simulator. Every related screen and event carries `billing_mode=demo`. Do not collect card details or describe a simulated payment as a real transaction.

Subscription state	Local behavior
Trial or active	Permit new work within configured plan and installation limits
Past due	Read existing results; pause new paid AI and browser jobs
Cancel scheduled	Retain current entitlement until the stored period end
Cancelled	Read retained results; reject new executions unless a free plan permits them

Entitlement checks

Enforce maximum projects, active schedules, browser concurrency, browser minutes, AI requests, and artifact storage. Check before accepting a job and again before dispatch. Queue depth and global installation limits also apply, even if a workspace plan allows more concurrency. Lowering a plan prevents new excess work while already authorized running jobs finish within their reserved limits.

Usage ledger

Reserve a bounded maximum for a requested operation and reconcile it to actual measured usage on completion. Each usage event has a unique operation key so retries cannot duplicate consumption records. Track AI usage reported by the provider, generation requests, browser wall time, and storage bytes separately. Unknown provider usage is marked unknown, not zero.

Product billing policy determines whether failed attempts count toward an allowance. Infrastructure cost is still recorded even if the customer allowance is credited. Do not infer a currency charge from token counts without the configured provider, rate, and effective date. Quota estimates are labeled estimates.

Payment provider adapter

The adapter exposes checkout initiation, subscription lookup, cancellation, signature validation, and event normalization. The UI never activates a subscription from a checkout redirect alone. Provider events are verified, deduplicated, and applied using transition checks; reconcile out of order events against current provider state. No real checkout can be enabled while the adapter is demo mode.

A real payment provider account and sandbox credentials are required before commercial validation. Live collection additionally needs the owner's commercial configuration. These credentials are unnecessary for the localhost demo, and none are created or claimed configured by this design document.

15 Scalability and capacity analysis

Scalability verdict

The proposed design can add API replicas and browser workers without changing case or result contracts. PostgreSQL remains the durable control store, and local file storage becomes shared object storage. The local Codex login remains a single operator adapter and is not a scalable shared inference service for customers. Hosted AI needs separately provisioned product credentials or an eligible user authorized integration.

Planning model rather than benchmark

Assume a case averages 20 seconds and usable slot utilization is 70 percent. Estimated throughput is slots multiplied by 3600, multiplied by 0.70, divided by 20. Assume 0.75 GiB of memory per active browser slot for initial sizing; actual pages may need substantially more.

Browser slots	Estimated cases per hour	Browser memory assumption
2 local	252	1.5 GiB
10 hosted pilot	1260	7.5 GiB
50 larger pool	6300	37.5 GiB

These figures exclude queue wait, AI drafting, expensive fixture setup, and target bottlenecks. They describe aggregate work capacity, not a latency guarantee. CPU, browser memory, and target throttling determine the usable worker count; adding API servers alone will not make tests run faster.

Artifact sizing

At 1000 cases per day, assume 2 MiB of retained evidence per case plus a 10 MiB trace for 10 percent of cases. Thirty days is approximately 87.9 GiB before metadata, replication, backups, or compression changes. Retention, per artifact limits, and failure focused trace collection matter more than keeping every event forever.

Scaling triggers

- Increase worker capacity when queue wait exceeds the target and CPU, memory, and target request budgets allow it; preserve per workspace fairness.
- Move artifacts to object storage before workers run on multiple hosts. Stream uploads; never buffer full traces inside an API process.
- Use bounded database pools, short transactions, composite indexes, and archived event partitions when measured query or vacuum pressure justifies them.
- If queue churn degrades product queries, isolate the job database or switch the QueuePort implementation. Keep immutable manifests and idempotency contracts unchanged.

Start with two global browser slots and one AI slot. A full laptop resource benchmark, representative page mix, and provider quota measurement are required before increasing these limits. No production capacity claim is justified by this arithmetic alone.

16 Verification and AI quality evaluation

Tests of the platform

Layer	Meaningful verification
Domain and contracts	State transitions, approval hashes, action schema, quota reservation, scope checks and verdict aggregation
Database integration	Cross tenant reads and writes, composite foreign keys, optimistic concurrency, transaction rollback and duplicate requests
Worker integration	Actual browser assertions, fixture ownership, interrupted jobs, bounded retries, cancellation and orphan recovery
AI integration	Authenticated real request, schema validation, source links, hallucinated targets, missing requirements and quota errors
End to end	Connect demo, discover, generate, approve, run, detect a defect, fix, rerun and export
Integration adapters	Local Git comparison, schedule fire deduplication, billing simulation and quota enforcement
Abuse and isolation	Blocked origins, redirects to internal targets, prompt injection, file path escape, secret exposure and duplicate events

Quality of generated tests

Create a fixed evaluation set of requirements with intended assertions and known ambiguities. Run generation on a controlled observation snapshot. Score each draft for requirement relevance, correct expectation, executable target, useful test data, redundancy, and honest handling of uncertainty. Record source and prompt versions for repeatability.

Invalid schema and invented secret references must be rejected every time. The maintainer grades semantic quality on the evaluation set before generalizing claims. Report the number reviewed and accepted, with material edits counted separately. Do not advertise an AI accuracy percentage from a handful of favorable examples.

Fault and durability tests

Interrupt a browser after a write, crash the dispatcher after job send but before acknowledgment, duplicate an API request, expire authentication, exhaust an AI quota, fail artifact upload, and force cleanup failure. Verify one logical run, preserved attempts, bounded resource use, truthful verdict, and recoverable fixture state.

Performance validation

Measure p50 and p95 API latency, queue wait, worker memory, browser duration, and database connections under the two slot local configuration. Target metadata API p95 below one second and UI progress within five seconds of an event. Run an unchanged fixture suite five times without unexplained result changes. These targets remain unverified until implementation measurements are saved.

17 ME presentation and completion gates

Demonstration application

Build an owned social web application with login, profile, posts, edit and delete permissions, search, and a simple comment flow. Use synthetic users and data. Provide build controlled defects such as unsaved edits, incorrect validation, and a missing ownership restriction in the tested flow. Defect switches are only for the owned demo target, never for customer applications.

Presentation sequence

- Show local health, the selected project, test roles, and actual Codex provider readiness.
- Connect the target, discover modules, enter requirements, and generate fresh scenario drafts through the authenticated provider.
- Inspect evidence and expected results, edit one draft, and approve an executable suite.
- Run the suite against the defective build and show real assertion failures with screenshots or trace evidence.
- Switch to the fixed build, verify its identity, rerun the same approved cases, and compare results.
- Show a local Git change analysis, a scheduled run, plan quotas, a labeled billing simulation, and a report export.

Completion checklist

Gate	Required evidence
Functional breadth	Every capability on page 2 has a working local path or a clearly marked unconfigured external dependency
Regression depth	At least 20 approved cases across five workflows; include happy path, validation and role checks
Real AI	One fresh successful generation with schema, model and usage metadata; no hardcoded cases masquerading as AI
Known defects	Each deliberately seeded defect covered by an approved assertion fails on the defective build and passes when fixed
Isolation and recovery	Cross tenant, duplicate run, cancellation, interrupted write and cleanup failure tests pass
Reproducibility	Setup guide, migrations, resettable demo data, version pins and saved evaluation results

Presentation fallback

Before presenting, verify internet, login, provider quota, available browser binaries and the local database. Keep prior real run reports and a recorded walkthrough available. Label these as historical results if used. Existing test execution must work without a new AI request; a network outage must not produce simulated live generation.

18 Build sequence and release migration

Implement in the following order and verify each stage before widening scope. The owner has delegated routine decisions; seek input only for missing access, a material scope tradeoff, or an external service action that needs the owner's account.

Stage	Exit condition
1 Runtime foundation	Dedicated PostgreSQL, migrations, owner setup, roles, projects, queue and installation health work locally
2 Discovery and AI	Controlled demo connection, bounded discovery, real Codex drafting, source links and versioned review pass
3 Regression engine	Restricted interpreter, fixtures, actual browser tests, evidence, history, cancellation and truthful outcomes pass
4 Full workflow	Local Git, build checks, schedules, quota ledger, billing simulator and exports work end to end
5 Presentation readiness	All page 18 gates pass; capacity observations and known limitations are recorded
6 Commercial hardening	Hosted provider access, payment sandbox, GitHub connection, secret store, object store and network isolation are validated

What changes for a hosted release

Move PostgreSQL to a managed service and local evidence to private object storage. Use a service secret store with rotation, HTTPS sessions, provider backed identity and account recovery, and persistent scheduler leadership. Scale API replicas and worker pools independently. Validate database backup restoration, artifact expiry, alerts and incident procedures before accepting customer projects.

Replace personal Codex login as the shared backend with product owned API credentials or another explicitly supported deployment integration. Sign in with ChatGPT for third party apps has eligibility and integration requirements; do not assume every commercial app can use a customer plan automatically. [9] Keep the provider contract so this change does not alter scenarios or the runner.

Highest implementation risks and resolutions

AI tool isolation must be verified against the installed CLI, not assumed from a read only flag. Localhost networking must be verified from the actual worker sandbox. Interrupted writes need fixture reconciliation. Queue limits need global enforcement, not only a per process counter. Hosted credential separation and destination enforcement are release requirements, even when a trusted local demo can run on one laptop.

Access needed

No cloud database account or separate OpenAI API key is required for the selected local architecture. Later external integrations need the owner's GitHub App configuration, payment provider sandbox, and approved hosted inference access. User authentication or payment setup cannot be supplied by an assistant owned account. Do not paste credentials into a design document or chat.

19 Decision record and technical references

Decisions retained for implementation

Choose PostgreSQL for relational integrity and a pg-boss queue to avoid a second local datastore. Choose a modular backend with separate workers to keep execution costs and failures away from API requests. Choose a restricted action interpreter and immutable approved cases to make results auditable. Choose local Codex only for trusted owner operated inference; retain a replaceable provider boundary for release.

Scalability review result: the design has a clear path to more workers and hosted services, with explicit bottlenecks and migration boundaries. Functional correctness, AI integration, network isolation, and measured capacity remain implementation gates. No load test or completed MVP is implied by this document.

Official and primary references

Checked on 8 October 2026. These references establish underlying capabilities. Numeric limits, data models, contracts, and scope choices are engineering decisions for this project.

[1] ChatGPT sign in, credential handling and authentication status.

[Codex authentication](#)

[2] Saved authentication, JSON events and schema constrained output.

[Codex noninteractive execution](#)

[3] Sandbox, approval, shell and web search configuration controls.

[Codex configuration reference](#)

[4] PostgreSQL backed jobs, transactions and worker orchestration.

[pg-boss project documentation](#)

[5] Row policies, default deny and roles that bypass row security.

[PostgreSQL row security](#)

[6] Browser runtime dependencies, version matching and container caveats.

[Playwright Docker guidance](#)

[7] Recorded browser actions and snapshots for investigating failures.

[Playwright Trace Viewer](#)

[8] Repository event subscriptions and webhook configuration.

[GitHub Apps and webhooks](#)

[9] Third party sign in, plan usage and eligibility boundaries.

[Sign in with ChatGPT quickstart](#)